

Core Language Changes for NB Comments at the November, 2019 (Belfast) meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4835](#).

RU007. [basic.life].8.3 Relax pointer value/aliasing rules

(This also addresses US047.)

1. Delete the note and example from 6.7.2 [intro.object]:

~~[Note: If the subobject contains a reference member or a const subobject, the name of the original subobject cannot be used to access the new object (6.7.3 [basic.life]). —end note] [Example:~~

```
struct X { const int n; };  
union U { X x; float f; };  
void tong() {  
    U u = {{ 1 }};  
    u.f = 5.f; //OK, creates new subobject of u (11.5 [class.union])  
    X *p = new (&u.x) X {2}; //OK, creates new subobject of u  
    assert(p->n == 2); //OK  
    assert(*std::launder(&u.x.n) == 2); //OK  
    assert(u.x.n == 2); //undefined behavior, u.x does not name new subobject  
}  
  
—end example]
```

2. Change 6.7.3 [basic.life] bullet 8.3 as follows:

If, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, a new object is created at the storage location which the original object occupied, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object, if:

- ...
 - the type of the original object is not const-qualified, and, if a class type, does not contain any non-static data member whose type is const-qualified or a reference type **neither a complete object that is const-qualified nor a subobject of such an object**, and
 - ...
-

US019, US020. Update ISO 9899 document reference from C11 to C17

1. Change 1 [intro.scope] paragraph 2 as follows:

C++ is a general purpose programming language based on the C programming language as described in ISO/IEC ~~9899:2011~~ **9899:2018** Programming languages – C (hereinafter referred to as the C standard). C++ provides...

2. Change 2 [intro.refs] bullet 1.5 as follows:

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

- ...
- ISO/IEC ~~9899:2011~~ **9899:2018**, Programming languages – C
- ...

3. Change 2 [intro.refs] paragraph 2 as follows:

The library described in Clause 7 of ISO/IEC ~~9899:2011~~ **9899:2018** is hereinafter called the C standard library.¹

CA038. Consider trailing *requires-clauses* for function identity

1. Merge 6.6 [basic.link] bullets 11.3 and 11.4 with the following changes:

Two names that are the same (Clause 6 [basic]) and that are declared in different scopes shall denote the same variable, function, type, template or namespace if

- ...
- when both names denote functions **or function templates**, the ~~parameter-type-lists of the functions (9.3.3.5 [dcl.fct]) are identical; and~~ **signatures (3.19 [defns.signature], 3.20 [defns.signature.templ]) are the same.**
- ~~when both names denote function templates, the signatures (13.7.6.1 [temp.over.link]) are the same.~~

2. Change 9.9 [namespace.udecl] paragraph 14 as follows:

If a function declaration in namespace scope or block scope has the same name and the same parameter-type-list (9.3.3.5 [dcl.fct]) as a function introduced by a *using-declaration*, and the declarations do not declare the same function, the program is ill-formed. If a function template declaration in namespace scope has the same name, parameter-type-list, **trailing requires-clause (if any)**, return type, and ~~template-parameter list~~ **template-head** as a function template introduced by a *using-declaration*, the program is ill-formed. [Note: Two *using-declarations*...

3. Change 9.9 [namespace.udecl] paragraph 14 as follows:

When a *using-declarator* brings declarations from a base class into a derived class, member functions and member function templates in the derived class override and/or hide member functions and member function templates with the same name, parameter-type-list (9.3.3.5 [dcl.fct]), **trailing requires-clause (if any)**, cv-qualification, and *ref-qualifier* (if any) in a base class (rather than conflicting). Such hidden

or overridden declarations are excluded from the set of declarations introduced by the *using-declarator*. [Example:...

4. No change to 9.11 [dcl.link] paragraph 5:

If two declarations declare functions with the same name and parameter-type-list (9.3.3.5 [dcl.fct]) to be members of the same namespace or declare objects with the same name to be members of the same namespace and the declarations give the names different language linkages, the program is ill-formed; no diagnostic is required if the declarations appear in different translation units. Except...

5. Change 12.2 [over.load] bullets 2.2 and 2.3 as follows:

Certain function declarations cannot be overloaded:

- Function declarations that differ only in the return type, the exception specification (14.5 [except.spec]), or both cannot be overloaded.
- Member function declarations with the same name, ~~and~~ the same parameter-type-list (9.3.3.5 [dcl.fct]), **and the same trailing requires-clause (if any)** cannot be overloaded if any of them is a static member function declaration (11.4.8 [class.static]). Likewise, member function template declarations with the same name, the same parameter-type-list, **the same trailing requires-clause (if any)**, and the same ~~template parameter lists~~ **template-head** cannot be overloaded if any of them is a static member function template declaration. The types of the implicit object parameters constructed for the member functions for the purpose of overload resolution (12.4.1 [over.match.funcs]) are not considered when comparing parameter-type-lists for enforcement of this rule. In contrast, if there is no static member function declaration among a set of member function declarations with the same name, ~~and~~ the same parameter-type-list, **and the same trailing requires-clause (if any)**, then these member function declarations can be overloaded if they differ in the type of their implicit object parameter. [Example:...
- Member function declarations with the same name, ~~and~~ the same parameter-type-list (9.3.3.5 [dcl.fct]), **and the same trailing requires-clause (if any)**, as well as member function template declarations with the same name, the same parameter-type-list, **the same trailing requires-clause (if any)**, and the same ~~template parameter lists~~ **template-head**, cannot be overloaded if any of them, but not all, have a *ref-qualifier* (9.3.3.5 [dcl.fct]). [Example:...

CZ044. Allow constexpr construct_at/destroy_at for automatic storage duration

Change 7.7 [expr.const] paragraph 5 as follows:

For the purposes of determining whether an expression **e** is a core constant expression, the evaluation of a call to a member function of `std::allocator<T>` as defined in 20.10.10.1 [allocator.members], where `T` is a literal type, does not disqualify the expression from being a core constant expression, even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression. Similarly, the evaluation of a call to `std::destroy_at`, `std::ranges::destroy_at`, `std::construct_at`, or `std::ranges::construct_at` is a valid core constant expression unless:

- for a call to `std::construct_at` or `std::ranges::construct_at`, the first argument, of type `T*`, does not point to storage allocated with `std::allocator<T>` **or to an object whose lifetime began within the evaluation of e**, or the evaluation of the underlying constructor call is not a core constant expression, or
- for a call to `std::destroy_at` or `std::ranges::destroy_at`, the first argument, of type `T*`, does not point to storage allocated with `std::allocator<T>` **or to an object whose lifetime began within the evaluation of e**, or the evaluation of the underlying destructor call is not a core constant expression.

US052. Non-executed return statements in coroutines

Change 8.7.4 [stmt.return.coroutine] paragraph 1 as follows:

A coroutine returns to its caller or resumer (9.5.4 [dcl.fct.def.coroutine]) by the `co_return` statement or when suspended (7.6.2.3 [expr.await]). A coroutine shall not ~~return to its caller or resumer by~~ **contain** a return statement (8.7.3 [stmt.return]). **[Note: For this determination, it is irrelevant whether the return statement is enclosed by a discarded statement (8.5.1 [stmt.if]). —end note]**

US053. Mandate the return type for `return_void` and `return_value` to be void

Change 8.7.4 [stmt.return.coroutine] paragraph 2 as follows:

The *expr-or-braced-init-list* of a `co_return` statement is called its operand. Let *p* be an lvalue naming the coroutine promise object (9.5.4 [dcl.fct.def.coroutine]). A `co_return` statement is equivalent to:

```
{ S; goto final-suspend; }
```

where *final-suspend* is the exposition-only label defined in 9.5.4 [dcl.fct.def.coroutine] and *S* is defined as follows:

- **If the operand is a braced-init-list or an expression of non-void type,** *S* is *p.return_value(expr-or-braced-init-list)*; ~~if the operand is a braced-init-list or an expression of non-void type;~~ **The expression *S* shall be a prvalue of type void.**
- **Otherwise,** *S* is **the compound-statement** `{ expressionopt ; p.return_void(); }`; ~~otherwise;~~ **The expression *p.return_void()* shall be a prvalue of type void.**

~~*S* shall be a prvalue of type void.~~

US065. Apply Coroutines TS issue 24 from P0664R8

Change 9.5.4 [dcl.fct.def.coroutine] paragraph 5 as follows:

A coroutine behaves as if its *function-body* were replaced by:

```
{  
    promise_type promise promise-constructor-arguments ;  
    try {  
        co_await promise.initial_suspend() ;  
    }  
    try { function-body  
    } catch ( ... ) {  
        if (!initial-await-resume-called)  
            throw ;  
        promise.unhandled_exception() ;  
    }  
    final-suspend :  
        co_await promise.final_suspend() ;  
}
```

where

- the *await-expression* containing the call to `initial_suspend` is the initial suspend point, and
- the *await-expression* containing the call to `final_suspend` is the final suspend point, and
- ***initial-await-resume-called* is initially `false` and is set to `true` immediately before the evaluation of *await-resume* (7.6.2.3 [expr.await]) of the initial suspend point, and**
- *promise-type* denotes the promise type, and
- ...

GB079. Add example for *private-module-fragment*

1. Delete the following production from the grammar in 6.6 [basic.link] paragraph 1:

~~*private-module-fragment*:~~
~~`module : private ; top-level-declaration-seqopt`~~

2. Delete 6.6 [basic.link] paragraph 2:

~~A *private-module-fragment* shall appear only in a primary module interface unit (10.1 [module.unit]). A module unit with a *private-module-fragment* shall be the only module unit of its module; no diagnostic is required.~~

3. Add the following as a new section following 10.4 [module.global]:

Private Module Fragment [module.private.frag]

***private-module-fragment*:**
`module : private ; top-level-declaration-seqopt`

A *private-module-fragment* shall appear only in a primary module interface unit (10.1 [module.unit]). A module unit with a *private-module-fragment* shall be the only module unit of its module; no diagnostic is required.

[Note: A *private-module-fragment* ends the primary module interface and commences an unimported implementation partition of the module. A *private-module-fragment* allows a module to consist of a primary interface and a single implementation partition without needing multiple translation units. The presence of a *private-module-fragment* affects:

- **the point by which the definition of an inline function declared in the module interface unit purview is required (9.2.7 [dcl.inline]),**
- **the point by which the definition of a function with a placeholder return type declared in the module interface unit purview is required (9.2.8.5 [dcl.spec.auto]),**
- **the instantiation contexts of templates instantiated before it (10.5 [module.context]), and**
- **the reachability of entities declared in the primary interface unit (10.6 [module.reach]).**

—end note]

[Example:

```

export module A;
inline void h(); // error: inline function h not defined
// before private module fragment

static void fn();
export struct X;
export void g(X *x) {
    fn(); // OK: call to static function in same TU
}
export X *factory(); // OK

module :private;
struct X {}; // definition not reachable from importers of A
X *factory() {
    return new X ();
}
void h() {}
void fn() {}

—end example

```

US087. Header unit imports cannot be cyclic, either

Change 10.4 [module.global] paragraph 9 as follows:

A translation unit has an *interface dependency* on a **module translation** unit U if it contains a *module-declaration* or *module-import-declaration* that imports U or if it has an interface dependency on a **module translation** unit that has an interface dependency on U. A translation unit shall not have an interface dependency on itself. [Example:...

US111. Constraint normalization and negation

Add the following as a new paragraph at the end of 13.5.1.1 [temp.constr.op] following paragraph 4:

[Note: A logical negation expression (7.6.2.1 [expr.unary.op]) is an atomic constraint; the negation operator is not treated as a logical operation on constraints. As a result, distinct negation *constraint-expressions* that are equivalent under 13.7.6.1 [temp.over.link] do not subsume one another under 13.5.4 [temp.constr.order]. Furthermore, if substitution to determine whether an atomic constraint is satisfied (13.5.1.2 [temp.constr.atomic]) encounters a substitution failure, the constraint is not satisfied, regardless of the presence of a negation operator. It is unclear in principle what the desired value is of a substitution failure in a negated concept-id; in the example below, does `requires !sad<typename T::type>` mean to require that there be no sad nested type, or that there be a nested type that is not sad? In effect it means the latter, whereas `requires !sad_nested_type<T>` means the former.

[Example:

```

template <class T> concept sad = false;

template <class T> int f1(T) requires (!sad<T>);
template <class T> int f1(T) requires (!sad<T>) && true;
int i1 = f1(42); // ambiguous, !sad<T> constraints are not formed from the same expression

template <class T> concept not_sad = !sad<T>;
template <class T> int f2(T) requires not_sad<T>;
template <class T> int f2(T) requires not_sad<T> && true;
int i2 = f2(42); // OK, !sad<T> constraints both come from not_sad

```

```
template <class T> int f3(T) requires (!sad<typename T::type>);
int i3 = f3(42);    // error, constraint not satisfied due to substitution failure

template <class T> concept sad_nested_type = sad<typename T::type>;
template <class T> int f4(T) requires (!sad_nested_type<T>);
int i4 = f4(42);    // OK, substitution failure contained within sad_nested_type
```

—end example] —end note]

[Drafting note: use of the term “concept-id” in this wording assumes the definition in the change to 13.3 [temp.names] for CA096.]

US132. Macros from the command-line not exported by header units

Change 15.4 [cpp.import] paragraph 3 as follows:

Each #define directive encountered when preprocessing each translation unit in a program results in a distinct *macro definition*. [Note: A predefined macro name (15.11 [cpp.predefined]) is not introduced by a #define directive. Implementations providing mechanisms to predefine additional macros are encouraged not to treat them as being introduced by a #define directive. —end note] Importing macros from a header unit...

US367. Instead of header inclusion, also permit header unit import

1. Change 6.7.5.4 [basic.stc.dynamic] paragraph 2 as follows:

...These implicit declarations introduce only the function names `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`. [Note: The implicit declarations do not introduce the names `std`, `std::size_t`, `std::align_val_t`, or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression*, or function call that refers to one of these functions without **importing or** including the header `<new>` (17.6.1 [new.syn]) is well-formed. However, referring to `std` or `std::size_t` or `std::align_val_t` is ill-formed unless the name has been declared by **importing or** including the appropriate header. —end note] Allocation and/or deallocation functions may also be declared and defined for any class (11.12 [class.free]).

2. Change 7.6.1.7 [expr.typeid] paragraph 6 as follows:

If the header `<typeinfo>` (17.7.2 [type.info]) is not **imported or** included prior to a use of `typeid`, the program is ill-formed.

3. Change 7.6.8 [expr.spaceship] paragraph 10 as follows:

The five comparison category types (17.11.2 [cmp.categories]) (the types `std::strong_ordering`, `std::strong_equality`, `std::weak_ordering`, `std::weak_equality`, and `std::partial_ordering`) are not predefined; if the header `<compare>` (17.11.1 [compare.syn]) is not **imported or** included prior to a use of such a class type – even an implicit use in which the type is not named (e.g., via the `auto` specifier (9.2.8.5 [dcl.spec.auto]) in a defaulted three-way comparison (11.11.3 [class.spaceship]) or use of the built-in operator) – the program is ill-formed.

4. Change 9.4.4 [dcl.init.list] paragraph 2 as follows:

...The template `std::initializer_list` is not predefined; if the header `<initializer_list>` is not **imported or** included prior to a use of `std::initializer_list` – even an implicit use in which the type is not named (9.2.8.5 [dcl.spec.auto]) – the program is ill-formed.

CA378. Remove constrained non-template functions

(This also addresses US095, US109, and CA110.)

1. Change 7.5.4 [expr.prim.id] paragraph 5 as follows:

A program that refers explicitly or implicitly to a function with a trailing *requires-clause* whose *constraint-expression* is not satisfied, other than to declare it, is ill-formed. [Example:

```
template<typename T> struct A {
    static void f(int) requires false;
}

void g() {
    A<int>::f(0); // error: cannot call f
    void (*p1)(int) = A<int>::f; // error: cannot take the address of f
    decltype(A<int>::f)* p2 = nullptr; // error: the type decltype(A<int>::f) is invalid
}
```

2. Change 9.3 [dcl.decl] paragraph 4 as follows:

The optional *requires-clause* (Clause 13 [temp]) in an *init-declarator* or *member-declarator* shall not be present when the declarator does not declare a **templated** function (9.3.3.5 [dcl.fct]). When present after a declarator, the *requires-clause* is called the trailing *requires-clause*. The trailing *requires-clause* introduces the *constraint-expression* that results from interpreting its *constraint-logical-or-expression* as a *constraint-expression*. [Example:

```
void f1(int a) requires true; // OK error: non-templated function
template<typename T>
    auto f2(int T a) -> bool requires true; // OK
template<typename T>
    auto f3(int T a) requires true -> bool; // error: requires-clause precedes trailing-return-type
void (*pf)() requires true; // error: constraint on a variable
void g(int (*)()) requires true; // error: constraint on a parameter-declaration

auto* p = new void(*) (char) requires true; // error: not a function declaration
```

3. Change 13.5.2 [temp.constr.decl] paragraphs 1 and 3 as follows:

A template declaration (Clause 13 [temp]) or **templated** function declaration (9.3.3.5 [dcl.fct]) can be constrained by the use of a *requires-clause*. This allows...

Constraints can also be associated...

A ~~template's~~ **declaration's** associated constraints are defined as follows:...